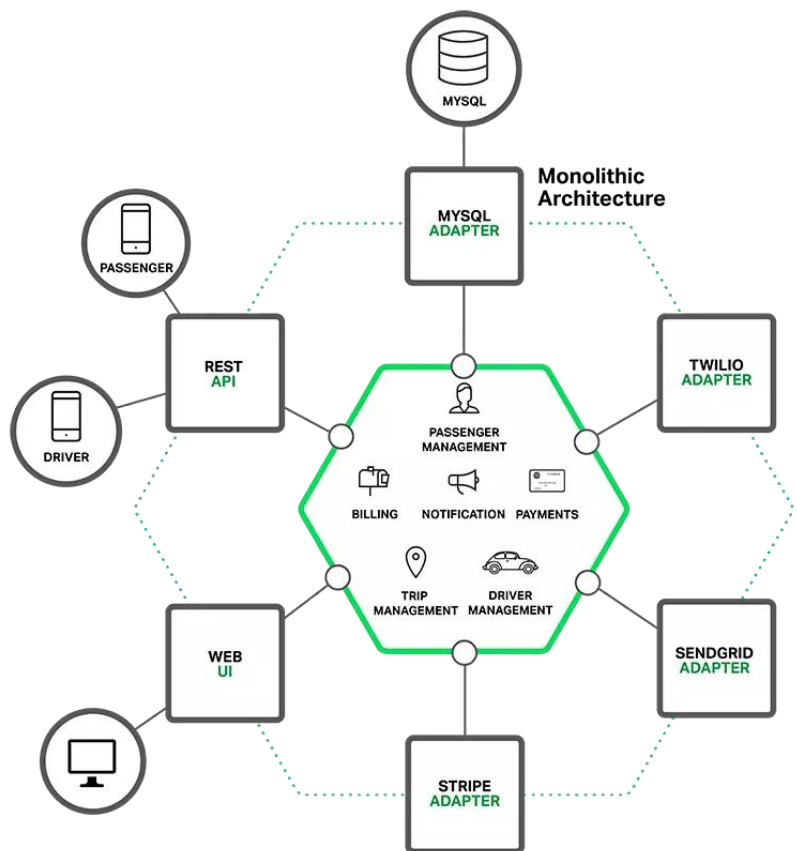


MICRO-SERVICES, MICRO-FRONTEND, MICRO-BACKEND

1. Micro-services

1.1. Kiến trúc một khối (Monolithic Architecture)

- **Monolithic Architecture** là một kiến trúc phần mềm truyền thống, trong đó toàn bộ ứng dụng được xây dựng như một đơn vị duy nhất, nguyên khối.
- Trong khối lớn ấy có chia thành các mô đun nhỏ, mỗi mô đun thực hiện một nhiệm vụ riêng và các mô đun thường gọi nhau qua function call.
- Tất cả các thành phần chức năng của ứng dụng (như giao diện người dùng, logic nghiệp vụ, lớp truy cập dữ liệu và các thành phần tích hợp bên ngoài) đều được **đóng gói lại**, có thể là một tệp thực thi (executable file) hoặc một tệp triển khai (deployable file) duy nhất (ví dụ: một tệp .war trong Java, một ứng dụng .exe trong C#, hoặc một thư mục ứng dụng Node.js lớn), chạy trong cùng một tiến trình (process), **chạy trên một server hoặc một môi trường** duy nhất.
- Kiến trúc này cũng đặc biệt phù hợp với các công ty outsource, họ có thể tạo ra một mẫu ứng dụng(template) và khi nhận được dự án với khách hàng mới họ chỉ việc bê mẫu này ra và chỉnh sửa, thêm thắt một vài phần khác biệt mà khách hàng yêu cầu.
- **Đặc điểm**
 - **Codebase thống nhất:** Tất cả mã nguồn (UI, logic nghiệp vụ, truy cập dữ liệu) **nằm trong một dự án duy nhất**.
 - **Triển khai đồng bộ:** Toàn bộ ứng dụng được **triển khai cùng một lúc**. Thay đổi bất kỳ phần nào (dù nhỏ) thường yêu cầu **rebuild và redeploy toàn bộ ứng dụng**.
 - **Cơ sở dữ liệu chung:** Thường sử dụng **một cơ sở dữ liệu duy nhất** để lưu trữ dữ liệu cho tất cả các chức năng.
 - **Chặt chẽ (tightly coupled):** Các thành phần của ứng dụng phụ thuộc lẫn nhau, ví dụ: thay đổi logic backend có thể ảnh hưởng đến frontend.
 - **Công nghệ đồng nhất:** Thường **sử dụng một stack công nghệ** (ngôn ngữ lập trình, framework) cho toàn bộ ứng dụng.
- *Hình bên minh họa Monolithic Architecture:* Một khối trung tâm duy nhất, là "khối nguyên khối" của ứng dụng. Trong khối này, **tất cả các chức năng cốt lõi được gom chung**:
 - *Passenger Management* (Quản lý hành khách)
 - *Billing* (Thanh toán)
 - *Notification* (Thông báo)



- *Payments (Thanh toán)*
- *Trip Management (Quản lý chuyến đi)*
- *Driver Management (Quản lý tài xế)*

Mặc dù được phân chia thành các "khu vực" chức năng, nhưng trong kiến trúc nguyên khối, tất cả các khu vực này thường được biên dịch và chạy như một tiến trình (process) hoặc một ứng dụng duy nhất. Chúng chia sẻ cùng một không gian bộ nhớ và tài nguyên. Tất cả những chức năng này được phát triển và triển khai cùng nhau như một phần của một ứng dụng lớn.

- **Các thành phần bên ngoài kết nối với khối trung tâm:**

- **"Adapter"** có nghĩa là một thành phần hoặc một lớp (layer) trong phần mềm có nhiệm vụ **thích nghi và cho phép một hệ thống hoặc một module giao tiếp với một hệ thống hoặc một dịch vụ khác mà ban đầu chúng không tương thích hoặc không được thiết kế để làm việc cùng nhau một cách trực tiếp.**
- Các adapter này là các thành phần cho phép khối nguyên khối tương tác với các hệ thống hoặc dịch vụ bên ngoài. Trong kiến trúc nguyên khối, các adapter này thường được nhúng trực tiếp vào trong ứng dụng hoặc được phát triển như một phần của codebase nguyên khối.
 - **REST API:** Cung cấp điểm truy cập cho các client di động (Passenger, Driver). Đây là lớp giao tiếp chính cho ứng dụng.
 - **Web UI:** Giao diện người dùng web cho người dùng máy tính để bàn.
 - **MySQL Adapter:**
 - Lớp kết nối với cơ sở dữ liệu MySQL.
 - Dịch các yêu cầu từ ứng dụng (ví dụ: lấy thông tin người dùng, lưu dữ liệu chuyến đi) thành các câu lệnh mà cơ sở dữ liệu **MySQL** có thể hiểu và thực thi.
 - Điều quan trọng là cơ sở dữ liệu này có thể chứa tất cả dữ liệu liên quan đến mọi chức năng trong khối trung tâm.
 - **Twilio Adapter, SendGrid Adapter, Stripe Adapter:** Các adapter này đại diện cho việc tích hợp với các dịch vụ bên thứ ba (SMS/gọi điện, email, thanh toán). Chúng cũng được nhúng hoặc kết nối trực tiếp vào ứng dụng nguyên khối.

- **Cách thức hoạt động (ví dụ):**

- Một hành khách (PASSENGER) sử dụng ứng dụng di động để đặt một chuyến đi.
- Yêu cầu này được gửi đến **REST API**.
- REST API chuyển yêu cầu vào bên trong khối nguyên khối.
- Khối nguyên khối xử lý yêu cầu, có thể liên quan đến các module như PASSENGER MANAGEMENT (để xác định hành khách), TRIP MANAGEMENT (để tạo chuyến đi), và PAYMENTS (để xử lý thanh toán ban đầu).
- Trong quá trình này, ứng dụng có thể tương tác với MYSQL ADAPTER để lưu thông tin chuyến đi vào cơ sở dữ liệu, và STRIPE ADAPTER để xử lý thanh toán.
- Sau khi chuyến đi được tạo, hệ thống có thể sử dụng NOTIFICATION để thông báo cho tài xế thông qua TWILIO ADAPTER (tin nhắn SMS) hoặc SENDGRID ADAPTER (email).
- Tài xế (DRIVER) nhận được thông báo, chấp nhận chuyến đi thông qua ứng dụng di động của mình (cũng qua REST API).

Tất cả logic xử lý việc chấp nhận chuyến, cập nhật trạng thái chuyến đi, và giao tiếp giữa hành khách/tài xế đều diễn ra bên trong cùng một khối nguyên khối.

- **Lợi ích**

- **Đơn giản trong giai đoạn đầu:** Dễ phát triển, kiểm thử và triển khai khi ứng dụng còn nhỏ.
- **Dễ quản lý:** Vì chỉ có một codebase, việc quản lý mã nguồn và cấu hình đơn giản hơn.
- **Hiệu suất tốt hơn trong một số trường hợp:** Do tất cả thành phần chạy trong cùng một tiến trình, không có độ trễ mạng như trong kiến trúc phân tán (microservice).
- **Dễ debug và kiểm thử:** Có thể chạy toàn bộ ứng dụng trên một máy local để kiểm tra → Gỡ lỗi dễ dàng hơn.

- **Thách thức**

- **Khó mở rộng (scalability):** Khi ứng dụng lớn lên, việc mở rộng (scale) đòi hỏi **nhân bản toàn bộ ứng dụng, dẫn đến lãng phí tài nguyên** (ví dụ: scale giao diện dù chỉ cần scale backend).
- **Khó bảo trì:** Codebase lớn dần trở nên phức tạp, khó hiểu, và **dễ xảy ra lỗi** khi thêm tính năng mới.
- **Phụ thuộc công nghệ:** **Khó nâng cấp hoặc thay đổi công nghệ** vì toàn bộ ứng dụng phụ thuộc vào một stack duy nhất.
- **Khó triển khai liên tục:** **Thay đổi nhỏ** cũng yêu cầu **triển khai lại toàn bộ ứng dụng**, làm chậm quy trình phát triển.
- **Khả năng phục hồi thấp:** Lỗi ở một thành phần (ví dụ: memory leak) có thể làm sập toàn bộ ứng dụng.
- **Hạn chế làm việc nhóm:** Các đội phát triển phải làm việc trên cùng một codebase, dễ gây xung đột và chậm tiến độ.

Vì những nhược điểm này, đặc biệt là với các ứng dụng lớn, phức tạp và yêu cầu khả năng mở rộng cao, các kiến trúc phân tán như Microservices đã trở nên phổ biến hơn.

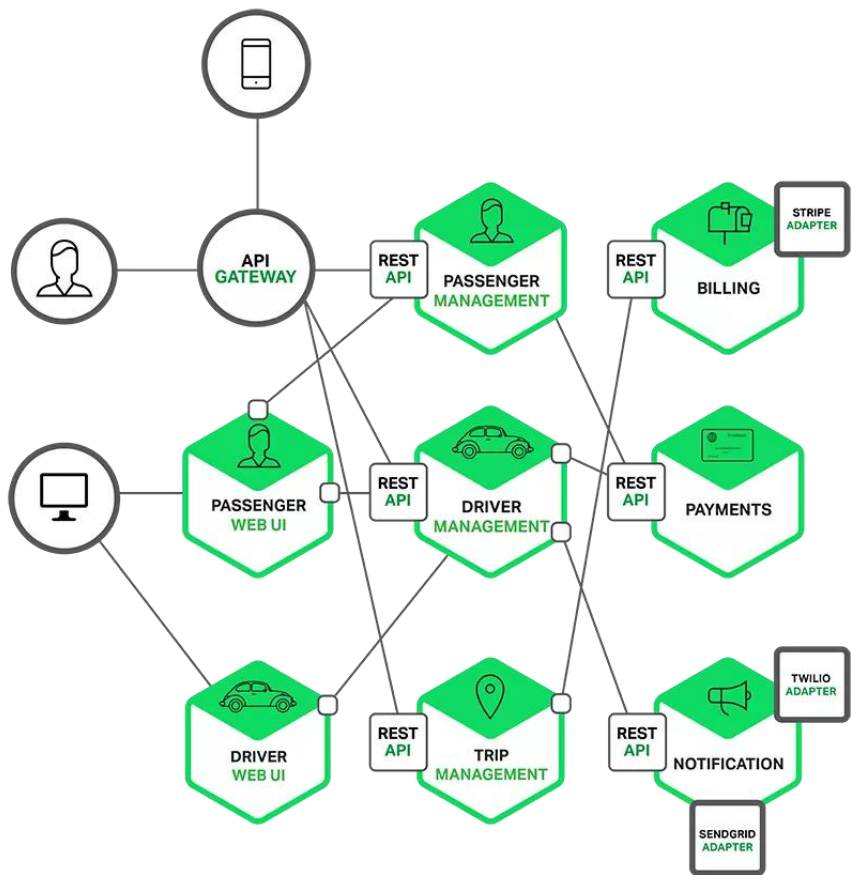
- **Khi nào sử dụng Monolithic Architecture?**

- **Ứng dụng nhỏ hoặc vừa:** Khi yêu cầu chức năng không quá phức tạp, đội phát triển nhỏ, và không cần mở rộng quy mô lớn.
- **Dự án khởi đầu (MVP):** Dễ xây dựng và triển khai nhanh để kiểm tra ý tưởng.
- **Đội ngũ hạn chế:** Khi không có đủ nhân lực hoặc kinh nghiệm để quản lý kiến trúc phân tán như microservice.
- **Yêu cầu hiệu suất cao:** Khi cần giảm thiểu độ trễ giao tiếp giữa các thành phần.

1.2. Kiến trúc dịch vụ nhỏ (Microservices Architecture)

- Microservice là một **kiến trúc phần mềm** trong đó **một ứng dụng lớn được chia thành nhiều dịch vụ nhỏ, độc lập**. Ý tưởng là chia nhỏ ứng dụng lớn ra thành các dịch vụ nhỏ kết nối với nhau.
- **Mỗi dịch vụ này đảm nhận một chức năng nghiệp vụ cụ thể**, có codebase riêng, có thể được phát triển, triển khai và bảo trì một cách **riêng biệt** bởi các nhóm phát triển nhỏ.
 - Khi nói mỗi **microservice có codebase riêng**, điều đó nghĩa là mỗi dịch vụ nhỏ (ví dụ: dịch vụ quản lý người dùng) sẽ có một tập hợp mã nguồn hoàn toàn độc lập với dịch vụ khác (ví dụ: dịch vụ xử lý đơn hàng). Chúng không chia sẻ cùng một thư mục gốc hay cùng một kho lưu trữ mã nguồn chung. Điều này giúp các nhóm phát triển có thể làm việc trên dịch vụ của họ mà không ảnh hưởng đến mã nguồn của các dịch vụ khác.

- Các microservice thường giao tiếp với nhau thông qua các giao thức nhẹ như **HTTP/REST** hoặc hệ thống hàng đợi tin nhắn (**message queue**).
- Mỗi vùng chức năng giờ được thực thi bởi một dịch vụ nhỏ. Ứng dụng web cũng có thể chia nhỏ hơn chuyên cho từng đối tượng người dùng (hành khách/tài xế). Thiết kế giao diện cho từng đối tượng người dùng giúp tối ưu trải nghiệm tốt hơn, tốc độ nhanh hơn, dễ tương thích hơn trong khi chức năng tối giản hơn.
- Thay vì một khối lượng giác lớn chứa tất cả, giờ đây chúng ta có nhiều khối lượng giác nhỏ hơn, mỗi khối đại diện cho một dịch vụ riêng biệt, độc lập:



- *PASSENGER MANAGEMENT: Quản lý riêng thông tin hành khách.*
 - *DRIVER MANAGEMENT: Quản lý riêng thông tin tài xế.*
 - *TRIP MANAGEMENT: Quản lý riêng các chuyến đi.*
 - *BILLING: Xử lý riêng các tác vụ lập hóa đơn.*
 - *PAYMENTS: Xử lý riêng các giao dịch thanh toán.*
 - *NOTIFICATION: Xử lý riêng việc gửi thông báo.*
- **REST API riêng cho từng dịch vụ:** Mỗi microservice có một REST API riêng. Điều này có nghĩa là các microservice này có thể được phát triển, triển khai và quản lý độc lập với nhau.
- **Tích hợp Adapter riêng:**
 - BILLING tích hợp với STRIPE ADAPTER (có thể là để xử lý các khoản thanh toán định kỳ hoặc liên quan đến hóa đơn).
 - NOTIFICATION tích hợp với TWILIO ADAPTER và SENDGRID ADAPTER.
 - Điều này cho thấy mỗi dịch vụ chỉ chịu trách nhiệm cho các tích hợp cần thiết của riêng nó, giảm sự phụ thuộc chéo.
- **Không thấy Cơ sở dữ liệu tập trung (MySQL) trực tiếp.** Điều này ngụ ý rằng:
 - Mỗi microservice có thể có cơ sở dữ liệu riêng của nó (ví dụ: PASSENGER MANAGEMENT có DB hành khách riêng, DRIVER MANAGEMENT có DB tài xế riêng). Đây là một đặc điểm cốt lõi của microservices để đảm bảo tính độc lập dữ liệu.
 - Hoặc, MySQL Adapter có thể được nhúng vào một hoặc nhiều microservice cụ thể nếu chúng cần truy cập dữ liệu quan trọng và không được chia sẻ rộng rãi.
- **Sự xuất hiện của Micro Frontends (MicroFE) - Các khối giao diện người dùng riêng biệt:** PASSENGER WEB UI, DRIVER WEB UI.

- **API Gateway:**

- Là một máy chủ (server) hoặc một lớp (layer) đứng giữa các ứng dụng client (ví dụ: ứng dụng web, ứng dụng di động, ứng dụng desktop) và một tập hợp các Microservices backend. Nó đóng vai trò là **điểm vào duy nhất (single entry point) cho tất cả các yêu cầu từ client đến các dịch vụ backend.**
- Thay vì client phải biết và gọi trực tiếp đến nhiều Microservices khác nhau, chúng chỉ cần gọi đến API Gateway. API Gateway sau đó sẽ chịu trách nhiệm định tuyến các yêu cầu đó đến đúng Microservice, tổng hợp kết quả và trả về cho client.
- **API Gateway** đóng vai trò là "cảnh sát giao thông" ở cửa ngõ. Tất cả các yêu cầu từ ứng dụng di động (Hành khách, Tài xế) và giao diện web (Passenger Web UI, Driver Web UI) đều đi qua API Gateway.
- **Lợi ích của API Gateway:**
 - **Định tuyến yêu cầu:** API Gateway sẽ định tuyến yêu cầu từ client đến microservice phù hợp dựa trên URL, header hoặc các tham số khác của yêu cầu.
 - **Tổng hợp phản hồi:** Một yêu cầu từ client có thể cần dữ liệu từ nhiều Microservice khác nhau. API Gateway có thể gửi đồng thời nhiều yêu cầu đến các Microservice backend, tổng hợp các phản hồi và gửi lại một phản hồi duy nhất cho client. Điều này giúp giảm số lượng "vòng lặp" (round trips) giữa client và backend, cải thiện hiệu suất cho client.
 - **Xác thực và ủy quyền:** Xử lý các vấn đề bảo mật như xác thực người dùng và ủy quyền trước khi chuyển yêu cầu đến các microservice. Điều này giúp các Microservice không cần phải tự xử lý logic xác thực và ủy quyền phức tạp. *Ví dụ: Khi một người dùng đăng nhập, API Gateway sẽ xác thực token của họ. Nếu hợp lệ, nó cho phép yêu cầu đi tiếp; nếu không, nó sẽ từ chối truy cập.*
 - **Giới hạn tốc độ, giám sát, ghi nhật ký:**
 - Kiểm soát số lượng yêu cầu mà một client có thể gửi trong một khoảng thời gian nhất định để ngăn chặn lạm dụng hoặc tấn công từ chối dịch vụ (DoS).
 - Thu thập dữ liệu về các yêu cầu và phản hồi (ví dụ: thời gian phản hồi, mã trạng thái HTTP) để giám sát hiệu suất và phát hiện lỗi.
 - Ghi nhật ký tất cả các yêu cầu đi qua, cung cấp một cái nhìn tổng quan về luồng giao tiếp.
 - **Giảm sự phức tạp cho client:** Client không cần biết về vị trí hay số lượng của các microservice.

- **Luồng hoạt động (ví dụ):**

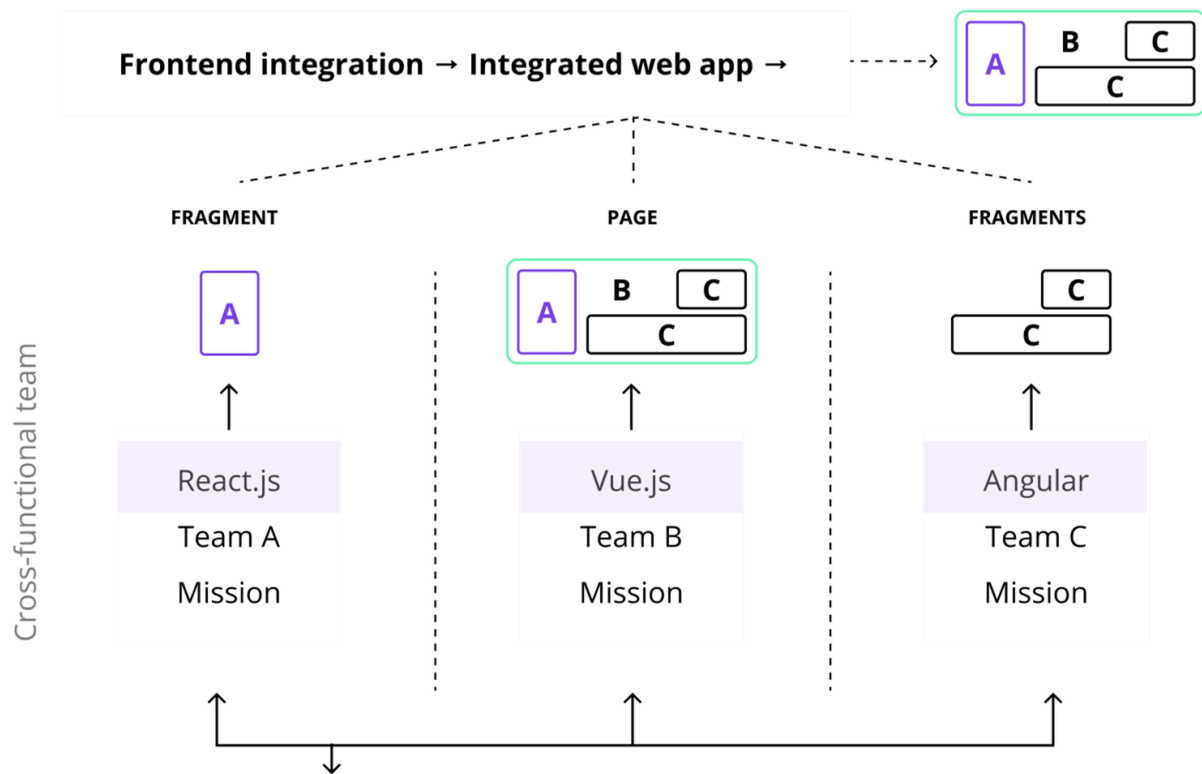
- Một hành khách (PASSENGER) sử dụng ứng dụng di động để đặt một chuyến đi.
- Yêu cầu này được gửi đến **API GATEWAY.**
- API GATEWAY nhận yêu cầu, có thể xác thực người dùng, sau đó **định tuyến yêu cầu đến các microservice phù hợp**, ví dụ:
 - PASSENGER MANAGEMENT để xác định hành khách và thông tin của họ.
 - TRIP MANAGEMENT để tạo và quản lý chuyến đi.
 - PAYMENTS để xử lý giao dịch thanh toán.
- Các microservice này có thể giao tiếp với nhau (ví dụ: TRIP MANAGEMENT có thể gọi đến DRIVER MANAGEMENT để tìm tài xế phù hợp, hoặc gọi đến NOTIFICATION để gửi thông báo).

- PAYMENTS sẽ tương tác với cổng thanh toán (không hiển thị adapter trực tiếp nhưng ngụ ý tích hợp).
- NOTIFICATION sẽ sử dụng TWILIO ADAPTER hoặc SENDGRID ADAPTER để gửi tin nhắn/email.
- Cuối cùng, **API GATEWAY tổng hợp các phản hồi từ các microservice và gửi lại cho ứng dụng di động** của hành khách.
- **Mỗi Microservice trong hệ thống đặt xe này có thể:**
 - **Viết bằng ngôn ngữ lập trình khác nhau:** PassengerService có thể được viết bằng **Node.js** (do yêu cầu về tốc độ phát triển và xử lý I/O cao); PaymentService có thể dùng **Java** (vì tính ổn định, bảo mật cao, phù hợp với các hệ thống tài chính)...
 - **Sử dụng cơ sở dữ liệu (Database) riêng biệt:** PassengerService có thể dùng **MongoDB** để lưu trữ hồ sơ người dùng linh hoạt; DriverService có thể dùng **PostgreSQL** để quản lý thông tin tài xế và các dữ liệu liên quan đến lịch sử hoạt động một cách có cấu trúc...
 - **Triển khai (Deploy) riêng biệt, không ảnh hưởng lẫn nhau:**
 - Khi có một lỗi hoặc cần cập nhật chức năng "đăng ký" trong PassengerService, chỉ cần triển khai lại PassengerService mà không ảnh hưởng đến khả năng "đặt xe" của TripService hay "thanh toán" của PaymentService.
 - Nếu NotificationService gặp vấn đề với việc gửi SMS qua Twilio, các chức năng cốt lõi khác của ứng dụng như đặt xe, quản lý tài xế vẫn hoạt động bình thường, chỉ có phần thông báo bị ảnh hưởng.
 - Việc mở rộng hệ thống trở nên dễ dàng hơn: nếu có quá nhiều chuyến đi được tạo ra, ta chỉ cần tăng số lượng instance (máy chủ) của TripService mà không cần phải mở rộng toàn bộ hệ thống.
- **Đặc điểm nổi bật:**
 - **Độc lập:** Mỗi microservice hoạt động độc lập, có thể được triển khai, cập nhật hoặc sửa lỗi mà không ảnh hưởng đến các dịch vụ khác trong hệ thống.
 - **Tập trung vào một nghiệp vụ:** Mỗi microservice chỉ chịu trách nhiệm cho một chức năng cụ thể (ví dụ: quản lý người dùng, xử lý đơn hàng, quản lý sản phẩm).
 - **Công nghệ đa dạng:** Các microservice có thể được phát triển bằng các ngôn ngữ lập trình và công nghệ khác nhau tùy theo yêu cầu của từng dịch vụ.
 - **Khả năng mở rộng độc lập:** Khi một dịch vụ gặp tải cao, chỉ cần mở rộng dịch vụ đó mà không cần mở rộng toàn bộ ứng dụng.
 - **Phân tán quản lý dữ liệu:** Mỗi microservice thường có cơ sở dữ liệu riêng, đảm bảo tính độc lập và giảm thiểu sự phụ thuộc.
- **Ưu điểm**
 - Cho phép dễ dàng continuous delivery và deployment các ứng dụng lớn, phức tạp:
 - Cải thiện khả năng bảo trì - mỗi service tương đối nhỏ do đó dễ hiểu và thay đổi hơn
 - Khả năng testing dễ dàng hơn - các services nhỏ hơn và nhanh hơn để test
 - Khả năng triển khai tốt hơn - các services có thể được triển khai độc lập
 - Cho phép các services được phát triển bởi những team khác nhau. Mỗi team có thể phát triển, thử nghiệm, triển khai và mở rộng quy mô dịch vụ của mình một cách độc lập với tất cả các team khác.

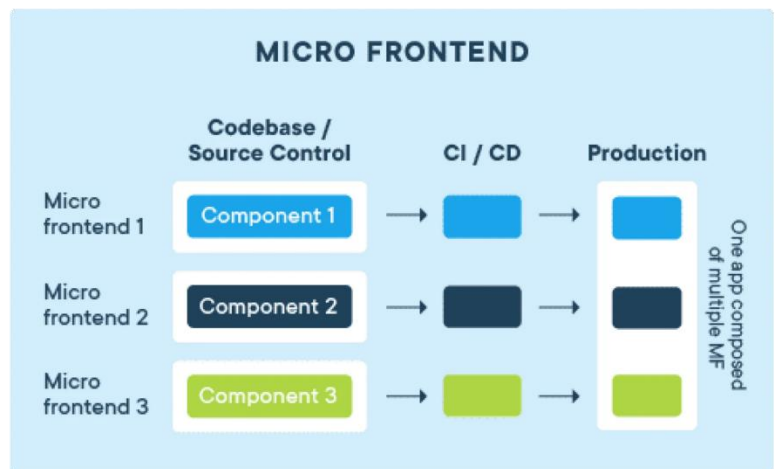
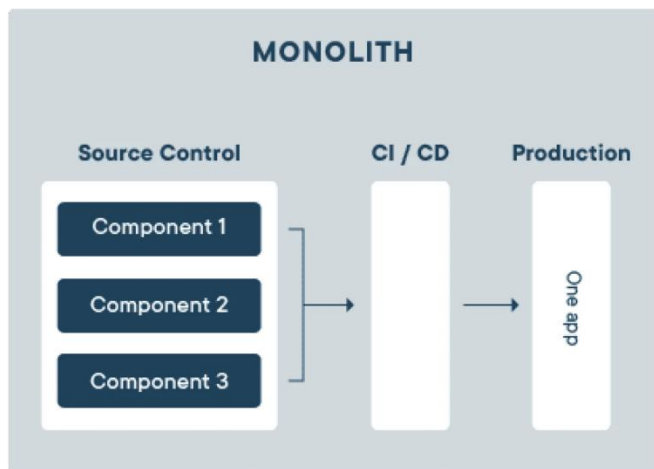
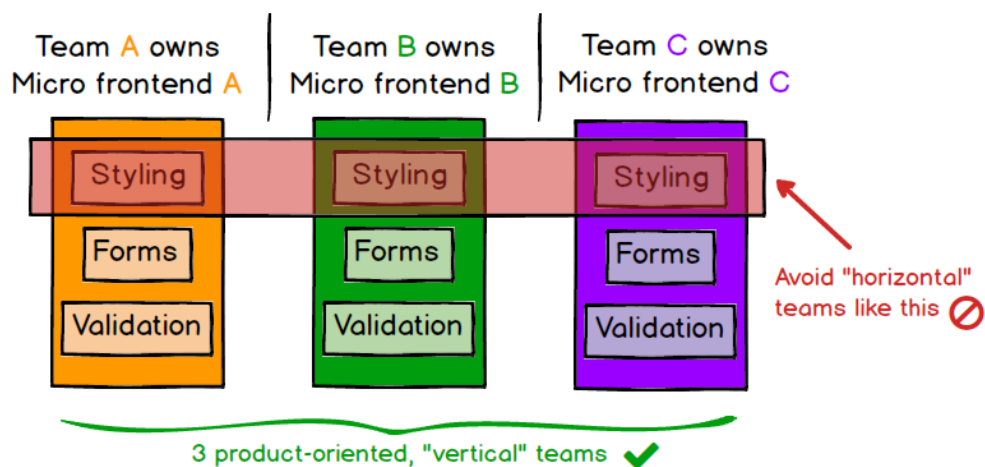
- Giảm thiểu rủi ro: Nếu có lỗi trong một service thì chỉ có service đó bị ảnh hưởng. Các services khác sẽ tiếp tục xử lý các yêu cầu. Trong khi đó, một thành phần hoạt động sai của kiến trúc một khối có thể làm ảnh hưởng toàn bộ hệ thống.
- Dễ dàng thay đổi sử dụng các công nghệ mới: Khi triển khai các services bạn có thể lựa chọn nhiều công nghệ mới. Tương tự khi có thay đổi lớn đối với các services hiện có bạn có thể dễ dàng thay đổi công nghệ.
- **Nhược điểm**
 - Các nhà phát triển phải đối phó với sự phức tạp của việc tạo ra một hệ thống phân tán:
 - Cần implement việc communication giữa các inter-services
 - Handle partial failure là rất phức tạp vì một luồng xử lý cần đi qua nhiều services
 - Việc thực hiện các requests trải rộng trên nhiều services khó khăn hơn, điều này cũng đòi hỏi sự phối hợp cẩn thận giữa các teams
 - Khó khăn trong việc đảm bảo toàn vẹn CSDL nếu triển khai theo kiến trúc cơ sở dữ liệu phân vùng
 - Triển khai và quản lý microservices nếu làm thủ công theo cách đã làm với ứng dụng một khối phức tạp hơn rất nhiều
 - Phải xử lý sự cố khi kết nối chậm, lỗi khi thông điệp không gửi được hoặc thông điệp gửi đến nhiều đích đến vào các thời điểm khác nhau
- Microservices không chỉ về chia tách các services sẵn có trong monolithic. Điều quan trọng chính là nhìn vào các tính năng trong một ứng dụng monolithic, ta có thể **nhận biết, xác định các yêu cầu và khả năng cần thiết để đáp ứng một nghiệp vụ**. Sau đó **từng nghiệp vụ này sẽ được xây dựng thành những service nhỏ, độc lập**. Những services này có thể sử dụng các nền tảng công nghệ khác nhau và phục vụ một mục đích cụ thể và có giới hạn.

2. Micro Frontend

- Micro Frontend là một kiến trúc phần mềm áp dụng các **nguyên tắc của microservice** vào phần giao diện người dùng (frontend).
- Thay vì xây dựng một ứng dụng frontend nguyên khối (monolithic frontend), Micro Frontend chia nhỏ giao diện người dùng thành nhiều "micro frontend" nhỏ hơn, độc lập.
- Mỗi micro frontend này chịu trách nhiệm cho một phần cụ thể của giao diện người dùng hoặc một tính năng riêng biệt (ví dụ: một widget, một module sản phẩm, một giỏ hàng).
- Các MicroFE được tích hợp để tạo thành một giao diện hoàn chỉnh.



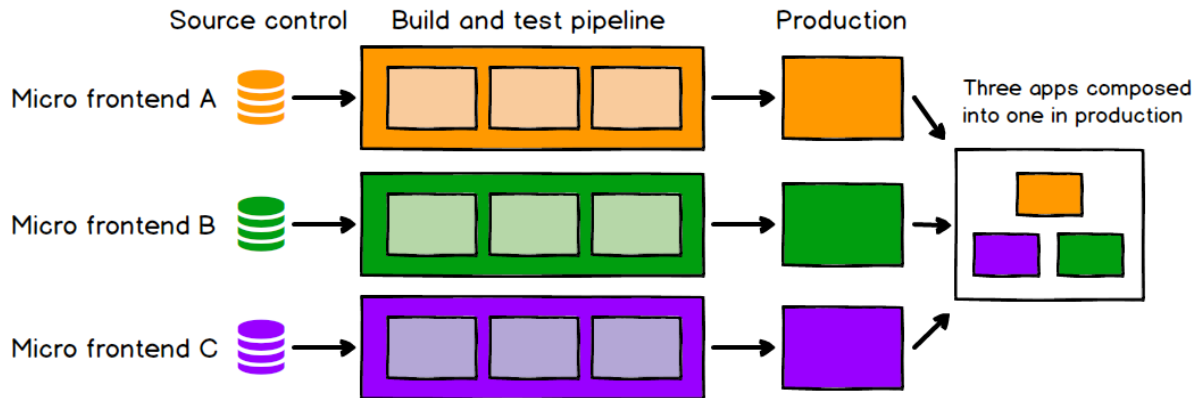
VERTICALLY ARRANGED TEAMS AND SYSTEMS



- **CI/CD** là viết tắt của **Continuous Integration** (Tích hợp liên tục) và **Continuous Delivery** hoặc **Continuous Deployment** (Triển khai liên tục). Đây là một tập hợp các phương pháp và công cụ trong phát triển phần mềm nhằm tự động hóa và cải thiện quá trình tích hợp, kiểm thử, và triển khai mã nguồn, giúp tăng tốc độ phát triển và đảm bảo chất lượng sản phẩm.
- **Continuous Integration (CI - Tích hợp liên tục)**
 - CI là quá trình tự động tích hợp mã nguồn **từ nhiều nhà phát triển vào một kho lưu trữ mã nguồn chung** (repository) **thường xuyên, thường là nhiều lần trong ngày**. Mỗi lần tích hợp, mã nguồn sẽ được kiểm thử tự động để phát hiện lỗi sớm.
 - **Đặc điểm:**
 - **Tích hợp thường xuyên:** Các nhà phát triển đẩy (commit) mã nguồn lên repository (ví dụ: GitHub, GitLab) nhiều lần mỗi ngày.
 - **Kiểm thử tự động:** Mỗi lần commit, hệ thống CI sẽ tự động chạy các bài kiểm thử (unit tests, integration tests) để đảm bảo mã mới không gây lỗi.
 - **Phát hiện lỗi sớm:** Giúp tìm và sửa lỗi ngay trong giai đoạn phát triển, giảm chi phí sửa lỗi sau này.
 - **Công cụ phổ biến:** Jenkins, GitLab CI, GitHub Actions, CircleCI, Travis CI.
 - **Quy trình CI:**
 1. Nhà phát triển commit mã nguồn lên repository.
 2. Hệ thống CI (như Jenkins) phát hiện thay đổi và kích hoạt pipeline.
 3. Pipeline thực hiện:
 - **Build ứng dụng** (biên dịch mã nguồn).
 - **Chạy các bài kiểm thử tự động** (unit tests, integration tests).
 - **Báo cáo kết quả** (thành công hay thất bại).
 4. Nếu có lỗi, đội phát triển được thông báo để sửa.
 - **Lợi ích:**
 - Giảm xung đột mã nguồn khi nhiều người cùng làm việc.
 - Phát hiện lỗi sớm, tránh tích lũy lỗi lớn.
 - Đảm bảo mã nguồn luôn ở trạng thái có thể triển khai.
- **Continuous Delivery (CD - Triển khai liên tục)**
 - **là bước mở rộng của CI**, trong đó mã nguồn sau khi được tích hợp và kiểm thử tự động sẽ được tự động triển khai đến một môi trường **staging hoặc production-like (gần giống production)**. Tuy nhiên, việc **triển khai lên production thường yêu cầu phê duyệt thủ công**.
 - **Môi trường production** là môi trường cuối cùng trong chuỗi phát triển phần mềm, nơi mã nguồn sau khi được phát triển, kiểm thử (qua các môi trường như development, testing, staging) được triển khai để hoạt động chính thức. Đây là nơi ứng dụng "sống" và cung cấp dịch vụ cho người dùng thực tế, chẳng hạn như khách hàng truy cập một trang web thương mại điện tử hoặc người dùng sử dụng một ứng dụng di động.
 - **Môi trường staging** là một môi trường thử nghiệm được thiết kế để mô phỏng gần giống với môi trường production. Đây là bước cuối cùng trước khi triển khai mã nguồn lên production, nơi các tính năng, bản cập nhật hoặc sửa lỗi được kiểm tra trong điều kiện gần giống thực tế.

- **Môi trường production-like** là một thuật ngữ chung chỉ bất kỳ môi trường nào được cấu hình để mô phỏng gần giống với môi trường production. Staging là một dạng phổ biến của production-like environment.
- **Đặc điểm:**
 - **Tự động hóa đến môi trường staging:** Mã nguồn được build, kiểm thử, và triển khai đến môi trường thử nghiệm (staging) một cách tự động.
 - **Phê duyệt thủ công cho production:** Quá trình triển khai lên production thường **cần sự xác nhận từ đội phát triển hoặc quản lý**.
 - **Đảm bảo chất lượng:** Chỉ mã nguồn đã qua kiểm thử mới được triển khai.
 - **Công cụ:** Tương tự CI (Jenkins, GitLab CI, GitHub Actions), kết hợp với các công cụ triển khai như Docker, Kubernetes, hoặc Ansible.
- **Quy trình CD:**
 1. Sau khi CI hoàn tất (build và test thành công), pipeline tiếp tục triển khai mã nguồn đến môi trường staging.
 2. **Chạy các kiểm thử bổ sung** (như performance tests, security tests) trong môi trường staging.
 3. Đội phát triển **xem xét và phê duyệt để triển khai lên production** (nếu cần).
 4. **Cập nhật ứng dụng** cho người dùng cuối.
- **Lợi ích:**
 - Tăng tốc độ triển khai, giảm thời gian đưa tính năng mới đến người dùng.
 - Giảm rủi ro khi triển khai nhờ kiểm thử tự động.
 - Đảm bảo ứng dụng luôn ở trạng thái sẵn sàng triển khai.
- **Continuous Deployment (Triển khai liên tục)**
 - Là một bước xa hơn Continuous Delivery, mức độ tự động hóa cao nhất của CI/CD, trong đó mọi thay đổi mã nguồn vượt qua kiểm thử tự động sẽ được tự động triển khai lên môi trường production mà không cần phê duyệt thủ công.
 - **Đặc điểm:**
 - **Tự động hóa hoàn toàn:** Mã nguồn từ commit đến production không cần can thiệp thủ công.
 - **Yêu cầu kiểm thử nghiêm ngặt:** Cần hệ thống kiểm thử rất mạnh để đảm bảo không có lỗi nghiêm trọng khi triển khai.
 - **Phù hợp với ứng dụng lớn:** Thường được sử dụng bởi các công ty lớn với quy trình DevOps trưởng thành (như Netflix, Amazon).
 - **Quy trình Continuous Deployment:**
 1. Tương tự CI/CD, mã nguồn được commit, build, và kiểm thử tự động.
 2. Nếu tất cả kiểm thử thành công, mã nguồn được tự động triển khai lên production.
 3. Theo dõi và giám sát (monitoring) để phát hiện vấn đề sau triển khai.
 - **Lợi ích:**
 - Tính năng mới được triển khai nhanh chóng, đôi khi chỉ vài phút sau khi commit.
 - Giảm thiểu công việc thủ công, tăng hiệu quả đội phát triển.
 - Phù hợp với các ứng dụng cần cập nhật liên tục (như ứng dụng SaaS).
 - **Thách thức:**
 - Yêu cầu hệ thống kiểm thử rất mạnh và đáng tin cậy.
 - Cần giám sát chặt chẽ môi trường production để phát hiện lỗi ngay lập tức.

- Có thể không phù hợp với các ứng dụng yêu cầu kiểm soát nghiêm ngặt (như hệ thống tài chính).



- Mỗi Micro-Frontend có **pipeline CI/CD riêng**, cho phép build, kiểm thử, và triển khai độc lập mà không cần redeploy Host Container hoặc các Micro-Frontend khác.
- Mã nguồn của Micro-Frontends được **lưu trữ trên các server hoặc CDN riêng**, cho phép **tải động từ các vị trí khác nhau**.
- Cập nhật Micro-Frontend chỉ cần **đẩy mã mới lên server/CDN**, **Host Container sẽ tự động tải phiên bản mới nhất khi người dùng truy cập**.

Versioning: Mỗi Micro-Frontend có thể được đánh phiên bản (versioning) để đảm bảo tương thích với Host Container.

Rollback: Nếu một Micro-Frontend gặp lỗi, có thể rollback về phiên bản cũ mà không ảnh hưởng đến toàn bộ ứng dụng.

CDN Optimization: Sử dụng CDN để giảm độ trễ khi tải Micro-Frontend từ các server khác nhau.

- Tại sao CI/CD quan trọng trong Microservices, nơi có rất nhiều dịch vụ nhỏ độc lập (và phát triển phần mềm hiện đại)?
 - **Độc lập triển khai:** Mỗi microservice có thể có chu trình CI/CD riêng, cho phép các nhóm phát triển triển khai dịch vụ của họ một cách độc lập mà không ảnh hưởng đến các dịch vụ khác.
 - **Tăng tốc độ phát triển:** Với nhiều dịch vụ và nhiều nhóm làm việc, CI/CD giúp tự động hóa quá trình tích hợp và triển khai, tránh tắc nghẽn.
 - **Giảm độ phức tạp:** Tự động hóa giúp quản lý sự phức tạp của việc xây dựng và triển khai hàng chục hoặc hàng trăm microservice.
 - **Chất lượng cao hơn:** Phát hiện và giải quyết vấn đề sớm trong chu trình phát triển.
- **Đặc điểm nổi bật:**
 - **Chia nhỏ giao diện người dùng:** Tách biệt các phần của UI thành các đơn vị độc lập.
 - **Phát triển độc lập:** Mỗi micro frontend có thể được phát triển, triển khai và quản lý bởi một nhóm riêng biệt, sử dụng các framework và công nghệ khác nhau nếu cần.
 - **Tích hợp linh hoạt:** Các micro frontend được tích hợp lại với nhau để tạo thành một ứng dụng frontend hoàn chỉnh. Việc tích hợp có thể thông qua JavaScript modules, Web Components, hoặc các framework chuyên dụng.
 - **Giảm thiểu sự phụ thuộc:** Thay đổi ở một micro frontend ít có khả năng ảnh hưởng đến các micro frontend khác.

- **Lợi ích:**
 - **Phân chia trách nhiệm:** Các nhóm có thể sở hữu và làm việc trên các phần UI cụ thể.
 - **Tăng tốc độ phát triển:** Giảm thiểu xung đột giữa các nhóm và tăng tốc độ phát hành tính năng.
 - **Linh hoạt công nghệ:** Cho phép sử dụng các công nghệ frontend khác nhau trong cùng một ứng dụng.
 - **Nâng cấp dần dần (Incremental Upgrades):** Có thể nâng cấp hoặc thay thế từng phần của giao diện người dùng mà không cần viết lại toàn bộ.
- **Các thành phần chính của kiến trúc Micro-Frontend:**
 - **Host container (hay còn gọi là Shell Application hoặc Container App):**
 - Tất cả các micro-Frontend bắt đầu từ một host container. Khi người dùng gõ URL của ứng dụng vào trình duyệt, **Host Container là thứ đầu tiên được tải về**. Nó là "cánh cổng" dẫn vào toàn bộ ứng dụng.
 - Thường là **một file JavaScript nhỏ gọn (và đôi khi là một file HTML cơ bản) được tải về đầu tiên**. File này chứa
 - Các thành phần giao diện chung (như header, footer, thanh điều hướng).
 - Logic để khởi tạo ứng dụng và quyết định Micro-Frontend nào sẽ được hiển thị dựa trên URL hoặc trạng thái của ứng dụng.
 - Hãy hình dung Host Container như một **khung sườn chính của một ngôi nhà** hoặc **trang web trống rỗng ban đầu** khi bạn truy cập. Nhiệm vụ của nó là **cung cấp cấu trúc cơ bản và là điểm khởi đầu để "đính kèm" các phần nhỏ hơn (Micro-Frontends)** vào.
 - Thực hiện tải và quản lý các Micro-Frontend, cũng như điều phối giao tiếp giữa chúng, định nghĩa các khu vực trên trang mà các Micro-Frontend sẽ được tải vào.
 - Chịu trách nhiệm tải mã nguồn của Micro-Frontend cần thiết và dỡ bỏ các Micro-Frontend không còn được sử dụng để tối ưu hiệu suất.
 - Những thành phần mang tính toàn cục như footer, header và điều hướng trang thường được tích hợp vào host container, để đảm bảo tính thống nhất về giao diện và trải nghiệm người dùng (UI/UX consistency).
 - Ngoài ra, Host Container có thể quản lý các dịch vụ chung như:
 - **Routing:** Điều hướng giữa các Micro-Frontend.
 - **State Management:** Quản lý trạng thái toàn cục (ví dụ: Redux, Vuex).
 - **Authentication/Authorization:** Xử lý đăng nhập hoặc xác thực người dùng.
 - **Ví dụ:** Trong một trang thương mại điện tử, Host Container có thể là ứng dụng chính tải header (với logo và thanh tìm kiếm), footer, và menu điều hướng. Nó cũng quyết định khi nào hiển thị Micro-Frontend cho danh mục sản phẩm hoặc giỏ hàng.
 - Host Container không nhất thiết phải chứa toàn bộ giao diện chung. Trong một số trường hợp, các thành phần như header/footer có thể được tách thành Micro-Frontend riêng để tăng tính module hóa.
 - Đôi khi, Host Container cũng quản lý các thư viện, CSS, hoặc assets dùng chung để tránh trùng lặp tài nguyên giữa các Micro-Frontend.

○ **Micro-Frontends:**

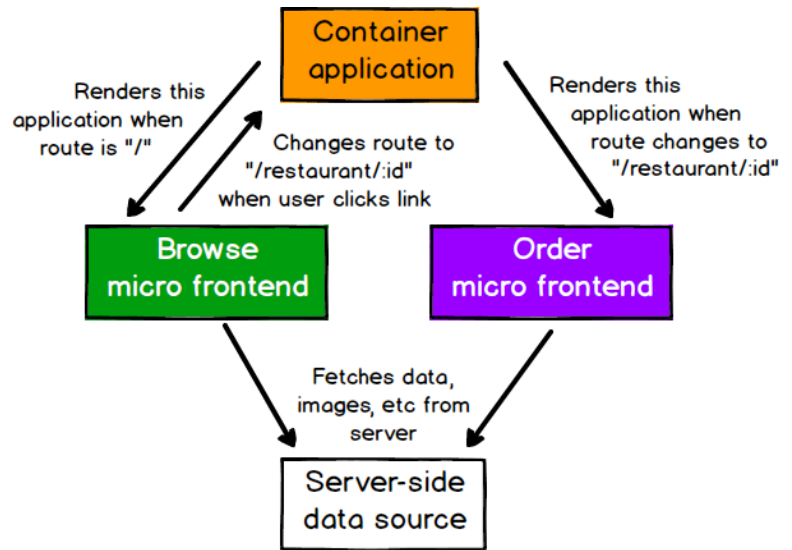
- Một host container sẽ có nhiều Micro Frontend - là những **mô đun độc lập chịu trách nhiệm cho một tính năng cụ thể hoặc một phần** của ứng dụng.
- Chúng được phát triển, triển khai, và quản lý độc lập, tương tự như cách microservice hoạt động → giảm sự phụ thuộc lẫn nhau, cho phép các đội phát triển làm việc song song.
- Nếu Host Container là khung sườn của ngôi nhà, thì **Micro-Frontends chính là các phòng chức năng riêng biệt** trong ngôi nhà đó (ví dụ: phòng khách, nhà bếp, phòng ngủ). Mỗi phòng có công năng riêng, được trang trí theo cách riêng, và có thể được xây dựng bởi một nhóm riêng biệt.
- Mỗi Micro-Frontend có:
 - **Codebase riêng:** Mã nguồn của nó nằm riêng biệt.
 - **Công nghệ riêng (khuyến khích):** Có thể được viết bằng React, Vue, Angular, hoặc thậm chí là Vanilla JavaScript. Điều này cho phép các nhóm lựa chọn công nghệ phù hợp nhất với tính năng của họ và giúp nâng cấp công nghệ từng phần.
 - **Đội ngũ phát triển riêng:** Mỗi đội chịu trách nhiệm hoàn toàn cho vòng đời của Micro-Frontend của mình.
 - **Triển khai riêng:** Mỗi Micro-Frontend có thể được triển khai độc lập, không cần triển khai lại toàn bộ ứng dụng khi có thay đổi.
- Chúng giao tiếp với Host Container hoặc các Micro-Frontend khác qua các cơ chế như:
 - **API:** Gọi các API backend hoặc giao tiếp với Micro-Frontend khác.
 - **Event Bus:** Sử dụng sự kiện (events) để truyền thông điệp.
 - **Shared State:** Quản lý trạng thái chung qua Redux, Context API, hoặc các công cụ tương tự.
- Mỗi MF chỉ quan tâm đến một bộ chức năng nhất định, giúp giảm độ phức tạp và tăng tốc độ phát triển.
- Nhóm phát triển MF thường chịu trách nhiệm cho cả giao diện người dùng và đôi khi cả phần backend tương ứng (Backend-for-Frontend - BFF) nếu có.
- Micro-Frontends không chỉ giới hạn ở giao diện mà còn có thể bao gồm logic nghiệp vụ phía client (client-side business logic).
- Chúng có thể được triển khai ở cả client-side (trong trình duyệt) hoặc server-side (kết hợp tại server trước khi gửi đến trình duyệt).

○ **Module Federation (Webpack):**

- Một tính năng của Webpack cho phép chia sẻ mã nguồn, thư viện, và thậm chí cả các thành phần UI giữa các ứng dụng JavaScript độc lập vào thời gian chạy (runtime).
- Thay vì đóng gói tất cả các thư viện dùng chung vào mỗi Micro-Frontend, Module Federation **cho phép một Micro-Frontend (hoặc Host Container) "export" một phần code của nó để các Micro-Frontend khác có thể "import" và sử dụng.**
- Giảm kích thước bundle của mỗi Micro-Frontend, tránh trùng lặp code, và cho phép chia sẻ tài nguyên hiệu quả hơn. Rất phổ biến trong các kiến trúc Micro-Frontend hiện đại.

○ Routing Mechanism

- Hệ thống định tuyến (routing) **quyết định Micro-Frontend** nào được hiển thị dựa trên URL hoặc hành động của người dùng.



▪ Chức năng:

• Client-side

Routing: Host Container sử dụng thư viện như React

Router hoặc Single-SPA để **chuyển đổi giữa các Micro-Frontend**.

- **Server-side Routing:** Trong trường hợp tích hợp server-side, **server quyết định Micro-Frontend** nào được gửi đến trình duyệt.

- **Ví dụ:** Khi người dùng truy cập /products, Host Container tải Micro-Frontend "Danh sách sản phẩm".

○ Shared Services / Libraries

- Các thư viện hoặc dịch vụ chung được **chia sẻ giữa Host Container và các Micro-Frontend** để đảm bảo tính thống nhất và giảm dư thừa mã nguồn.

▪ Chức năng:

- **UI Component Library:** Cung cấp **các thành phần giao diện chung** (như button, modal) để đảm bảo thiết kế đồng nhất.
- **State Management:** **Quản lý trạng thái chung** (ví dụ: thông tin người dùng, giỏ hàng) qua Redux, MobX, hoặc Context API.
- **Utility Functions:** **Các hàm chung như xử lý định dạng ngày giờ, gọi API, hoặc xác thực.**

- **Ví dụ:** Một thư viện UI như Material-UI được chia sẻ để tất cả Micro-Frontends sử dụng, đảm bảo các nút bấm hoặc giao diện có phong cách đồng nhất.

○ Communication Mechanism (Cơ chế giao tiếp)

- Cung cấp một cơ chế để các Micro-Frontend độc lập có thể giao tiếp với nhau để chia sẻ dữ liệu hoặc kích hoạt hành động.

▪ Các phương pháp:

- **Custom Events:** Sử dụng sự kiện trình duyệt (CustomEvent) để gửi thông điệp giữa các Micro-Frontend.
- **Event Bus:** Một hệ thống trung gian để truyền thông điệp (ví dụ: sử dụng thư viện như PubSubJS).
- **Shared State:** Sử dụng Redux hoặc Context API để quản lý trạng thái chung.
- **URL-based Communication:** Sử dụng query parameters hoặc hash trong URL để truyền dữ liệu.

- Vì các Micro-Frontend hoạt động độc lập, chúng không thể trực tiếp gọi hàm của nhau. Một "Event Bus" (hoặc một thư viện quản lý trạng thái chung như

Redux/Vuex được Host Container khởi tạo và chia sẻ) cho phép các Micro-Frontend phát ra sự kiện và lắng nghe sự kiện từ các Micro-Frontend khác.

- Cho phép các thành phần UI khác nhau phản ứng với các hành động xảy ra ở nơi khác trong ứng dụng.
- Ví dụ: khi sản phẩm được thêm vào giỏ hàng (từ Micro-Frontend sản phẩm), một sự kiện được gửi đến Micro-Frontend là giỏ hàng nhỏ trên header để cập nhật số lượng.

○ **Build and Deployment Pipeline**

- Mỗi Micro-Frontend thường có pipeline CI/CD riêng để xây dựng (build), kiểm thử, và triển khai độc lập.
- Tự động hóa quá trình xây dựng, kiểm thử, và triển khai từng Micro-Frontend và Host Container.
- Tương tự như Microservices, mỗi Micro-Frontend có CI/CD pipeline riêng. Các công cụ như Webpack, Parcel, Rollup được sử dụng để đóng gói mã nguồn. Các hệ thống CI/CD như Jenkins, GitLab CI, GitHub Actions sẽ tự động chạy các build script và triển khai các thành phần.
- **Chức năng:**
 - Tự động build và kiểm thử Micro-Frontend khi có thay đổi mã nguồn.
 - Triển khai Micro-Frontend lên CDN hoặc server riêng, sau đó được Host Container tải về.
- Đảm bảo quá trình phát triển và triển khai nhanh chóng, đáng tin cậy và tự động.

○ **API Gateway / Backend Integration**

- Micro-Frontends thường giao tiếp với backend (MicroBE hoặc các microservice) thông qua một **API Gateway** hoặc trực tiếp qua API.
- **Chức năng:**
 - API Gateway hợp nhất các yêu cầu từ nhiều Micro-Frontends, gửi đến backend phù hợp, và trả về dữ liệu.
 - Đảm bảo bảo mật (như xác thực token) và tối ưu hóa giao tiếp (như caching).
- **Ví dụ:** Trong một ứng dụng thương mại điện tử, Micro-Frontend "Thanh toán" gọi API Gateway để lấy thông tin phương thức thanh toán từ backend.

• **Cách hoạt động của Micro Frontend (Dynamic Runtime Integration)**

- **Người dùng gửi yêu cầu truy cập trang web (URL):** Trình duyệt gửi request đến máy chủ lưu trữ **host container** (hay app shell).
- **Host container được gửi về trước:** Đây là phần nhẹ, chỉ gồm HTML khung chính, các phần tử như layout, menu trống, router, loading placeholders...
- **Host container xác định phần nào cần được tải:** Nó sử dụng một **Module Loader** (như Single-SPA, Module Federation) để xác định Micro-Frontend nào cần tải dựa trên:
 - **URL:** Ví dụ, /products tải Micro-Frontend "Danh sách sản phẩm".
 - **Hành động người dùng:** Nhấp vào nút "Giỏ hàng" tải Micro-Frontend "Giỏ hàng".
 - **Điều kiện logic:** Ví dụ, chỉ tải Micro-Frontend "Thanh toán" khi người dùng đã đăng nhập.

- **Host gửi các lời gọi mạng (HTTP) để tải các micro-frontend:** Mỗi MFE là một bundle riêng biệt, được lấy về qua **CDN**, **server khác nhau**, hoặc **Module Federation**.
 - **Ví dụ:** Người dùng truy cập /cart trên trang web. Host Container gửi yêu cầu đến CDN để tải file JavaScript của Micro-Frontend "Giỏ hàng", sau đó render nó vào vùng nội dung chính.

Client-side loading: Micro-Frontends được tải động trong trình duyệt bằng JavaScript (ví dụ: thông qua import() trong Module Federation).

Server-side loading: Trong một số trường hợp, server có thể kết hợp các Micro-Frontend (server-side composition) trước khi gửi đến trình duyệt.

Lazy loading: Để tối ưu hóa hiệu suất, Micro-Frontends thường được tải lười (lazy loading), nghĩa là chỉ tải khi cần thiết.

- **Micro-frontend được nhúng vào host container tại runtime**
 - Micro-Frontend được render vào một vùng cụ thể trong giao diện (DOM) của Host Container
 - Dùng Webpack Module Federation, SystemJS, hoặc single-spa, các phần này được mount vào vùng hiển thị.
 - Host Container đảm bảo các Micro-Frontend hoạt động hài hòa, duy trì tính thống nhất về giao diện (UI consistency) và trải nghiệm người dùng (UX).
 - **Ví dụ:** Micro-Frontend "Danh sách sản phẩm" (viết bằng React) được render vào một <div> trong Host Container, trong khi Micro-Frontend "Giỏ hàng" (viết bằng Vue.js) được render vào một vùng khác.
- **MFE có thể giao tiếp với Host Container hoặc MFE khác**
 - Các Micro-Frontends giao tiếp với nhau hoặc với Host Container để chia sẻ dữ liệu hoặc kích hoạt hành động. Các phương pháp phổ biến:
 - **Custom Events:** Sử dụng CustomEvent trong trình duyệt để **gửi thông điệp** (ví dụ: thông báo "sản phẩm được thêm vào giỏ hàng").
 - **Event Bus:** Một hệ thống trung gian (như PubSubJS) để **truyền thông điệp giữa các Micro-Frontend**.
 - **Shared State:** Sử dụng Redux, Context API, hoặc một kho trạng thái chung.
 - **API Calls:** Micro-Frontend **gọi API từ backend** (thông qua API Gateway) để **lấy dữ liệu**.
 - Host Container có thể đóng vai trò trung gian, điều phối giao tiếp giữa các Micro-Frontend.
 - Giao tiếp cần được thiết kế cẩn thận để tránh phụ thuộc chặt chẽ (tight coupling) giữa các Micro-Frontend.
 - **Ví dụ:** Khi người dùng thêm sản phẩm vào giỏ hàng, Micro-Frontend "Danh sách sản phẩm" gửi một sự kiện đến Micro-Frontend "Giỏ hàng" để cập nhật số lượng.
- **Nếu người dùng chuyển hướng hoặc tương tác thêm:** Hệ thống sẽ tiếp tục tải các MFE cần thiết mà chưa từng tải trước đó (giống cơ chế "**lazy load**").
- **Kết thúc ứng dụng**
 - Khi người dùng thoát khỏi ứng dụng (đóng trình duyệt hoặc chuyển sang trang khác), các Micro-Frontend đang hoạt động có thể được hủy (unload) để giải phóng tài nguyên.

- **Host Container quản lý vòng đời (lifecycle) của Micro-Frontends**, đảm bảo hiệu suất tối ưu.
- Một số framework như Single-SPA hỗ trợ tự động unload Micro-Frontend khi không còn cần thiết, giảm sử dụng bộ nhớ.
- **Dữ liệu tạm thời** (như trạng thái giỏ hàng) có thể được lưu vào **local storage** hoặc **gửi về backend** trước khi thoát.

3. Micro-backend

- Micro Backend (MicroBE) là thuật ngữ dùng để **chỉ các microservice ở phía backend**, tức **các dịch vụ nhỏ, độc lập** chịu trách nhiệm xử lý logic nghiệp vụ, truy cập cơ sở dữ liệu, và cung cấp API cho frontend hoặc các hệ thống khác.
- Đây là một cách gọi khác của microservice, nhưng nhấn mạnh vào vai trò backend.
- Thuật ngữ Micro Backend thường không được sử dụng phổ biến như Microservice.
- Một mô hình liên quan là **Backend for Frontend (BFF)**, nơi **mỗi frontend (hoặc nhóm micro frontend) có một backend chuyên biệt của riêng nó** để phục vụ nhu cầu cụ thể của frontend đó, thay vì một backend tổng hợp duy nhất.
- **Đặc điểm:**
 - **Chức năng cụ thể:** Mỗi MicroBE xử lý một nghiệp vụ riêng, ví dụ: quản lý người dùng, xử lý đơn hàng, hoặc gửi thông báo.
 - **API-driven:** MicroBE cung cấp các API (REST, GraphQL, gRPC) để giao tiếp với frontend hoặc các microservice khác.
 - **Độc lập cơ sở dữ liệu:** Mỗi MicroBE thường có cơ sở dữ liệu riêng (database per service) để tránh phụ thuộc lẫn nhau.
 - **Tính phân tán:** MicroBE thường được triển khai trên các container (Docker) hoặc môi trường đám mây (AWS, Azure).
- **Lợi ích:**
 - **Tính module hóa:** Dễ dàng thêm, sửa, hoặc xóa một MicroBE mà không ảnh hưởng đến hệ thống còn lại.
 - **Khả năng mở rộng:** Có thể scale riêng từng MicroBE dựa trên tải (load) của chức năng đó.
 - **Độc lập đội phát triển:** Các đội backend có thể làm việc song song, sử dụng các công nghệ khác nhau.
 - **Khả năng phục hồi:** Nếu một MicroBE gặp lỗi, các MicroBE khác vẫn hoạt động bình thường.
- **Thách thức:**
 - **Quản lý dữ liệu:** Đảm bảo tính nhất quán dữ liệu (data consistency) giữa các MicroBE là một thách thức lớn, đặc biệt khi cần giao dịch phân tán (distributed transactions).
 - **Độ trễ mạng:** Giao tiếp giữa các MicroBE qua mạng có thể gây chậm trễ.
 - **Phức tạp DevOps:** Cần các công cụ như Kubernetes, service mesh (Istio, Linkerd) để quản lý và điều phối.
 - **Chi phí:** Vận hành nhiều MicroBE đòi hỏi chi phí cơ sở hạ tầng và nhân lực cao hơn.